

[← Go Back](#)

Hardware

Portenta Machine Control ▼

Tutorials

[Memory Partitioning for Use with the Arduino IDE](#)[Arduino Portenta Machine Control Library Guide](#)[Modbus RTU On Portenta Machine Control Using PLC IDE](#)[Modbus TCP On Portenta Machine Control Using PLC IDE](#)[Modbus TCP with Portenta Machine Control & Opta™](#)[Tank Thermoregulation with Portenta Machine Control & Opta™](#)[Connect an RTD/Thermocouple to the Portenta Machine Control](#)

Portenta Machine Control User Manual

Home / Hardware / Portenta Machine Control / **Portenta Machine Control User Manual**

Portenta Machine Control User Manual

Learn about the hardware and software features of the Arduino® Portenta Machine Control.

José Bagur,
Christopher Mendez,
Author Julián Caro Linares, and Taddy Chung
Last revision · 09/16/2025

PORTENTA
MACHINE
CONTROL
USER MANUAL



This user manual provides a comprehensive overview of the Portenta Machine Control, covering its major hardware and software elements. With this user manual, you will learn how to set up, configure, and use all the main features of the Portenta Machine Control.

ON THIS PAGE

Hardware and Software Requirements

[Hardware Requirements](#)[Software Requirements](#)[Portenta Machine Control Overview](#) +[First Use](#) +[Digital Outputs](#)[Analog Outputs](#) +[Digital Inputs](#)[Analog Inputs](#)[Programmable Digital I/O](#)[Communication](#) +[Real-Time Clock](#)[Temperature Measurements](#) +[Encoders](#)[Arduino Cloud](#)[Support](#) +

This User Manual teaches how to use the Portenta Machine Control with the new

`Arduino_PortentaMachineControl`

library. If you want to know how to use it with IEC 61131-3 PLC programming languages, check

 the [PLC IDE Tutorials](#). In case you are already using the older version of the

`Arduino_MachineControl`

library, check the following

[tutorial to know the differences and how to migrate your code to the latest version.](#)

Hardware and Software Requirements

Hardware Requirements

[Portenta Machine Control](#) (x1)

[Micro-USB cable](#) (x1)

+24 VDC/0.5 A power supply (x1)

2.4 GHz SMA male antenna (x1)
(only required for Wi-Fi®/Bluetooth® capabilities)

Software Requirements

[Arduino IDE 2.0+](#) or [Arduino Cloud Editor](#)

[Arduino_PortentaMachineControl](#) library



This User Manual shows how to use the Portenta Machine Control using the Arduino IDE environment. To learn more about how to use it with IEC-61131-3 languages and the PLC IDE, check out our tutorials [here](#).

Portenta Machine Control Overview

The Portenta Machine Control is designed for efficiency and adaptability in industrial environments. It is compatible with the Arduino framework and other embedded platforms and provides a flexible solution for controlling various equipment and machinery. The Portenta H7 board (included) is central to its operation, which ensures stable performance across a broad temperature spectrum, ranging from -40 °C to +85 °C, without external cooling requirements.



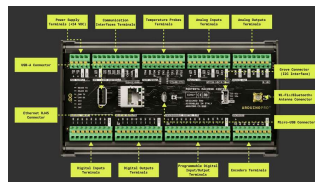
Portenta Machine Control board

This controller offers many connectivity options, from USB and Ethernet to Wi-Fi® and Bluetooth® Low Energy, as well as industry-specific protocols like Modbus and Canbus. It can also connect with various external sensors, actuators,

touch panels, showcasing its adaptability. It is designed for harsh industrial operations with features like DIN bar compatible housing, compact size, and an integrated Real-Time Clock (RTC). For real-time control or predictive maintenance tasks, the Portenta Machine Control is a solid choice for businesses aiming to enhance production and equipment management processes.

Portenta Machine Control Main Components

The Portenta Machine Control features a secure, certified, and durable design that enables it for automation and industrial applications.



Portenta Machine Control
main components

Here is an overview of the controller's main components shown in the image above:

Microcontroller: At the heart of the Portenta Machine Control is the STM32H747XI, a powerful, robust, and high-performance dual-core microcontroller from STMicroelectronics®. This microcontroller is built around an Arm® Cortex®-M7 and an Arm® Cortex®-M4 32-bit RISC cores. The Arm® Cortex®-M7 core operates at up to 480 MHz, while the Arm® Cortex®-M4 core operates at up to

Memory and storage:

2 MB of Flash Memory

1 MB of RAM

Additional onboard memory of
8 MB SDRAM

16 MB Flash QSPI

Security: The controller features an onboard ready-to-use secure element from NXP®, the SE0502. This secure element, specifically designed for Internet of Things (IoT) devices, provides advanced security features, perfect for Industrial IoT (IIoT) environments where security is critical.

Power architecture: The controller's power system was designed to be resilient. It operates at an input voltage of +24 VDC, with reverse polarity protection, ensuring the controller remains safeguarded from power irregularities.

Digital and analog ports: Equipped with a versatile set of input and output ports, the Portenta Machine Control supports:

8x digital input ports with 8x
Status LEDs labeled as

DIGITAL INPUTS

8x digital output ports with 8x
Status LEDs labeled as

DIGITAL OUTPUTS

3x software-configurable
analog input ports labeled as

ANALOG IN

4x analog output ports labeled
as

ANALOG OUT

12x digital programmable
input/output ports labeled as

PROGRAMMABLE DIGITAL I/O

Temperature sensing: With three software-configurable temperature channels, the Portenta Machine

Control can measure a variety of temperature ranges using:

- Type K thermocouples
- Type J thermocouples
- PT100 sensors

Communication interfaces and protocols: Seamless connectivity is a hallmark of this controller. The Portenta Machine Control offers high-speed, software-configurable communication interfaces and protocols such as:

- CAN bus
- RS-232
- RS-422
- RS-485
- I2C interface (accessible via a Grove connector)
- Modbus RTU (over RS-485)
- Modbus TCP (over Ethernet)

Ethernet and USB: The Portenta Machine Control features onboard Ethernet connectivity and full-speed USB-A and half-speed micro-USB Type B connectors for wired communication.

Wireless connectivity: The Portenta Machine Control supports 2.4 GHz Wi-Fi® (802.11 b/g/n) and Bluetooth® Low Energy (4.2 supported by firmware and 5.1 supported by hardware).

Additional features: The Portenta Machine Control features an onboard RTC with at least 48 hours of memory retention and two encoder channels. Moreover, Electrostatic Discharge (ESD) protection on all inputs and output ports ensures the longevity and durability of the controller.

rail, a grid, or a panel, providing quick and easy access to all input/output ports and peripherals.

Portenta Machine Control Core and Libraries

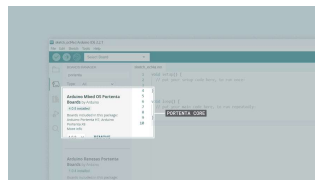
The

Arduino Mbed OS Portenta Boards

core contains the libraries and examples to work with Portenta's Machine Control peripherals and onboard components, such as its input ports, output ports, Wi-Fi® and Bluetooth® modules. To install the Portenta Machine Control core, navigate to **Tools > Board > Boards Manager** or click the **Boards Manager** icon in the left tab of the IDE. In the Boards Manager tab, search for `portenta` and install the latest

Arduino Mbed OS Portenta Boards

core version.



Installing the Arduino
Mbed OS Portenta Boards
core in the Arduino IDE

The

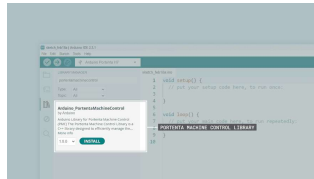
Arduino_PortentaMachineControl

library enables efficient management of the features of the Portenta Machine Control. To install the library:

1. Navigate to **Tools > Manage Libraries...** or click the **Library Manager** icon in the left tab of the IDE.

`portentamachinecontrol`
and install the latest
`Arduino_PortentaMachineControl`

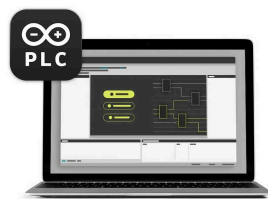
library version. If the IDE asks you to install additional dependent libraries, install all of them.



Installing the
Arduino_PortentaMachine
Control library in the
Arduino IDE

Arduino PLC IDE

PLC IDE is the Arduino solution to program Portenta Machine Control devices using the five programming languages recognized by the IEC 61131-3 standard.



Arduino PLC IDE

The IEC 61131-3 programming languages include:

- Ladder Diagram (LD)
- Functional Block Diagram (FBD)
- Structured Text (ST)
- Sequential Function Chart (SFC)
- Instruction List (IL)

In the PLC IDE, you can mix PLC programming with standard Arduino

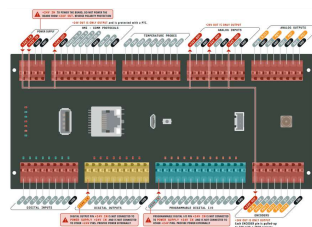
editor and share variables between the two environments. You can also automate tasks in your software applications; this gives you control over scheduling and repetition, enhancing the reliability and efficiency of your project. Moreover, communication protocols such as Modbus RTU and Modbus TCP can be managed effortlessly using integrated no-code fieldbus configurators.

Check out the following resources that will show you how to start with the Arduino PLC IDE and use IEC 61131-3 programming languages with the Portenta Machine Control:

[Arduino PLC IDE download page and tutorials](#)

[Portenta Machine Control tutorials](#)

Pinout



Portenta Machine Control pinout

The complete pinout is available and downloadable as PDF from the link below:

[Portenta Machine Control pinout](#)

Datasheet

The complete datasheet is available

link below:

[Portenta Machine Control
datasheet](#)

STEP Files

The complete STEP files are available and downloadable from the link below:

[Portenta Machine Control
STEP files](#)

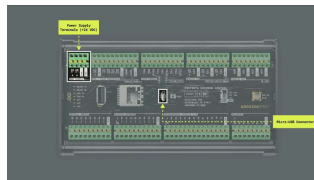
First Use

Powering the Portenta Machine Control

Portenta Machine Control can be powered in different ways:

Using an **external +24 VDC/0.5 A power supply** connected to Portenta's Machine Control power supply terminals. Please refer to the [pinout section](#) of the user manual.

Using a **micro-USB cable** (not included) for programming purposes only.

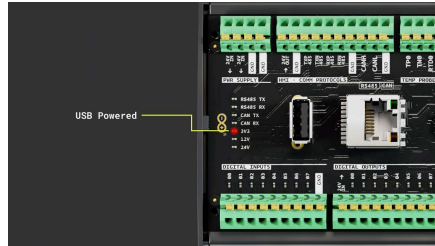


Different ways to power a Portenta Machine Control

The Portenta Machine Control has several LEDs that indicate how the board is being powered:

If the board is powered using a

The **12V LED** (yellow) and the **24V LED** (green LED) will be turned on if the board is powered using an **external +24 VDC power supply**.



Portenta Machine Control Terminals Features

Access to the main components and features of the Portenta Machine Control is provided through onboard terminal blocks, specifically from the SPTAF-1 connector series by Phoenix Contact. These connectors are distinguished by their low profile, ability to handle high currents, and intuitive push-in wire termination system. Below are the technical specifications of the Portenta Machine Control terminals:

Pitch: 3.5 mm

Connection method: Push-in spring connection

Connection direction: 45°

The connector and wire specifications for the Portenta Machine Control terminals are outlined in the table below, indicating supported conductor cross sections and their respective capacities:

Conductor Cross Section	Capacity
Solid	0.2 mm ² - 1.5 mm ²

Conductor Cross Section	Capacity
Flexible, with ferrule without plastic sleeve	0.25 mm ² - 0.75 mm ²
Flexible, with ferrule with plastic sleeve	0.25 mm ² - 0.75 mm ²
AWG	24 - 16

Hello World Example

Let's program the Portenta Machine Control with a modified version of the classic `hello world` example used in the Arduino ecosystem: the `Blink` sketch. This example will verify that the controller's connection to the Arduino IDE, its core functionalities, and the `Arduino_PortentaMachineControl` library are working as expected.

Remember to install the

Arduino Mbed OS
Portenta Boards

core and the



Arduino_PortentaMachineControl

library; please refer to [this section of the User Manual](#) to learn how to do it.

Copy and paste the sketch below into a new sketch in the Arduino IDE.


```

1 // Include the Arduino_Po
2 #include <Arduino_PortentaMachineControl_Digital>
3
4 void setup() {
5     // Initialize the dig
6     MachineControl_Digital
7 }
8
9 void loop() {
10    // Turn on the digital
11    MachineControl_Digital
12    delay(1000);
13    // Turn off the digital
14    MachineControl_Digital
15    delay(1000);
16 }

```

The sketch begins by including the `Arduino_PortentaMachineControl` library. The `setup()` function initializes the digital output terminals from this library. The `loop()` function, which continually runs after the `setup()` function is called, toggles a digital output at channel `0`.

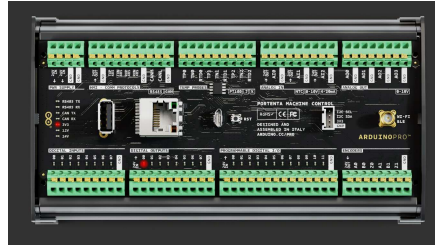
To upload the sketch to your Portenta Machine Control, click the **Verify** button to compile the sketch and check for errors; once verification is successful, click the **Upload** button to program the controller with the sketch.



Uploading a sketch to a Portenta Machine Control in the Arduino IDE

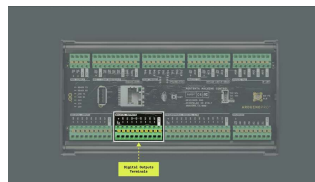
Upon successful upload, observe the red LED on your controller's digital

second, repeatedly. Notice that a **micro-USB cable** can power the Portenta Machine Control for this example.



Digital Outputs

The Portenta Machine Control has up to eight digital output channels, as shown in the image below.



Portenta Machine Control
digital output channels

Some of the key features of the digital output channels of the Portenta Machine Control are the following:

Digital outputs are **high-side switches**

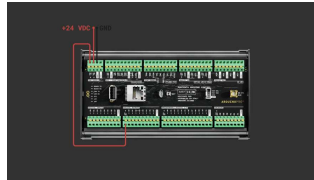
(TPS4H160AQPWPRQ1),
handling up to 500 mA.

All digital output terminals have **overcurrent protection**. If the current exceeds 700 mA (with a tolerance of $\pm 20\%$), the channel opens to prevent damage.

The digital output channels must be connected to an external +24 VDC power supply through pin `24V IN`; this

power supply can be shared with the controller's +24 VDC

i power supply, as shown in the image below. Moreover, pin `24V IN` is not galvanically isolated, meaning the input power supply voltage must share the same `GND` as the controller.



Powering the digital output channels of the Portenta Machine Control

There are two modes of overcurrent protection in the digital output channels:

- 1 **Latch mode:** When overcurrent is detected, the digital output channel remains open and can only be closed manually via software.
- 2 **Auto retry:** Upon detecting overcurrent, the channel opens. After a short duration (several tens of milliseconds), it attempts to close automatically. If the overcurrent condition persists in the channel, it will keep toggling.

Ensure each channel does not exceed a maximum current of

i 500 mA to avoid potential damage or malfunctions in the digital output channels.

The example sketch below showcases a "scanning" effect using the digital output channels of the Portenta Machine Control, activating each channel sequentially. This method also provides visual feedback through the Arduino IDE's Serial Monitor, indicating which channel is active at any given moment.

```

1  /*
2   Portenta Machine Control Example
3   Name: portenta_machine_control
4   Purpose: Demonstrates the use of the
5   @author Arduino PRO Core Team
6   @version 1.0 01/10/23
7  */
8
9  #include <Arduino_PortentaMachineControl.h>
10
11 void setup() {
12   // Initialize serial communication
13   Serial.begin(9600);
14
15   // Initialize the digital outputs
16   MachineControl_DigitalOutputs.begin(true);
17
18   // Turn all channels off
19   MachineControl_DigitalOutputs.turnOffAll();
20 }
21
22 void loop() {
23   // Sequentially activate each channel
24   for (int i = 0; i < 8; i++) {
25     // Turn on the current channel
26     MachineControl_DigitalOutputs.turnOn(i);
27     // Wait to make the channel active
28     Serial.println("- Ch" + String(i) + " is active");
29     delay(500);
30   }
31   // Turn all channels off
32   MachineControl_DigitalOutputs.turnOffAll();
33   delay(1000);
34 }

```

Note that the sketch shown above uses the following functions from the `Arduino_PortentaMachineControl` library:

```
MachineControl_DigitalOutputs.begin(true)
```

: This function initializes the digital outputs channels with overcurrent behavior set to

remain open until manually toggled in software.

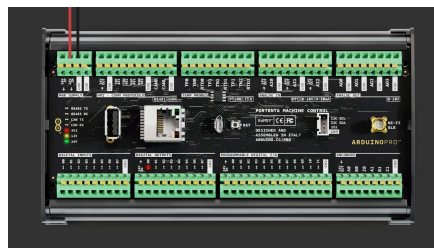
```
MachineControl_DigitalOutputs.writeAll(0)
```

: This function initially sets all digital output channels to an open state (off).

```
MachineControl_DigitalOutputs.write(channel,  
HIGH/LOW)
```

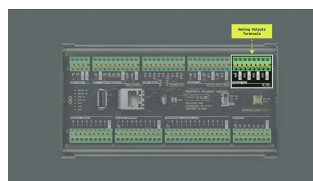
: This function controls individual channel states, turning them either on (**HIGH**) or off (**LOW**). In the example sketch, this function creates the "scanning" effect by activating one channel at a time.

The expected behavior of the digital output channels LEDs is shown below.



Analog Outputs

The Portenta Machine Control has up to four independent analog output channels, as shown in the image below. These analog output channels enable precise voltage control for various applications.



Portenta Machine Control
analog output channels

Some of the key features of the

Portenta Machine Control are the following:

Analog outputs can be configured with specific PWM periods, affecting the frequency and resolution of the voltage output.

Each channel supports voltage outputs ranging from 0 VDC to 10.5 VDC and can source up to 20 mA.

Each analog output channel is designed with a double low-pass filter and a high-current operational amplifier (OPA2990IDSGR) set up in a non-inverting topology with a gain factor of 3.3. This design allows for an effective filtering and amplification of the signal, resulting in a high-quality/low-noise analog output signal.

The output signal of the analog output channels of the Portenta



Machine Control is a DC voltage whose amplitude is a function of the defined PWM duty cycle.

Below is an example demonstrating using a single analog output channel to generate a sine wave voltage output.



```
1  /*
2   Portenta Machine Control
3   Name: portenta_machine_control
4   Purpose: This sketch controls a motor
5   using an analog output
6
7   @author Arduino PRO Core Team
8   @version 1.0 01/10/23
9   */
10
11  #include <math.h>
12  #include <Arduino_PortentaMachineControl.h>
13
14  // PWM period set to 4 ms
15  #define PERIOD_MS 4
16
17  void setup() {
18    // Initialize serial communication
19    Serial.begin(9600);
20
21    // Initialize the analog output
22    MachineControl_AnalogOutput(0);
23
24    // Set the PWM period
25    MachineControl_AnalogOutput(PWM_PERIOD);
26  }
27
28  void loop() {
```

In the example sketch, the sine wave signal is generated by iterating through 360 degrees; the sine function is computed for each degree value. The sine function yields a value between -1 and 1; to convert this into a voltage value between 0 and 10 VDC, an offset of 5 VDC is added, and the result is then multiplied by 5. With this formula, the sine wave oscillates between 0 to 10 VDC. The delay introduced at the end of each iteration helps adjust the frequency of the sine wave signal, resulting in the desired waveform at the analog output.

Notice that the sketch shown above uses the following functions from the

Arduino_PortentaMachineControl

```
MachineControl_AnalogOut.begin()
```

: This function initializes the analog output channels, preparing them for voltage output.

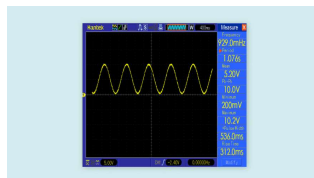
```
MachineControl_AnalogOut.setPeriod(channel, period)
```

: This function configures the PWM period for the specified analog output channel. In the example shown above, it is set to 4 ms or 250 Hz.

```
MachineControl_AnalogOut.write(channel, voltage)
```

: This function controls the voltage output for the specified channel. In the example above, a sine wave is generated for channel `A00` ranging from 0 to 10 VDC.

The expected result of the generated sine wave measured with an oscilloscope in the analog output channel `A00` is shown in the image below.



Generated sine wave using
analog output channel
A00 of the Portenta
Machine Control

Working With Analog Output Channel 2 (AO2)

Analog output channel 2 (AO2) of the Portenta Machine Control is connected to pin *PG7* on the Portenta H7, which features an **HRTIM (High-Resolution TIMer)** function. It is the only Analog Out pin on the Portenta Machine Control with an **HRTIM**

The *HRTIM* on *PG7* is mainly applicable for high-frequency PWM signals, which have constraints on the maximum period that can be achieved. With the availability of the *HRTIM_EEV2* function, which serves as an external event input, its configuration allows **AO2** to support high-resolution PWM signals with very short periods but also results in a constraint.

The *HRTIM* is configured with a frequency of **200 MHz** (tick time = **5 ns**), a clock prescaler set to **4**, and a maximum period of **0xFFFF** ticks

(**65533**  5 ns **4** = **1.31 ms**).

This configuration results in a maximum allowed period of **1.31 ms** for *AO2*.

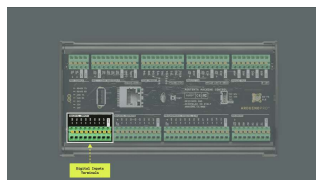
Knowing that the maximum allowable period for *AO2* is approximately **1.3 ms**, it makes it suitable only for high-frequency PWM signals. For applications requiring periods longer than *1.3 ms*, we recommend using other analog output channels, such as **Analog output channel 0 (AO0)**, **1 (AO1)**, or **3 (AO3)**, which can use standard timers. Please consider this characteristic when selecting the appropriate output channel for the development application.

The following code shows the setup of the *Analog Out* channels, including *AO2*. Please be aware that attempting to set a period longer than *1.3 ms* for *AO2* is not recommended.

```
1 #include <Arduino_Portenta_101.h>
2
3 #define PERIOD_MS_A02 1
4 #define PERIOD_MS 4
5
6 void setup() {
7   Serial.begin(9600);
8
9   MachineControl_AnalogOut0 = 0;
10  MachineControl_AnalogOut1 = 0;
11  MachineControl_AnalogOut2 = 0;
12  MachineControl_AnalogOut3 = 0;
13  MachineControl_AnalogOut4 = 0;
14  MachineControl_AnalogOut5 = 0;
15  MachineControl_AnalogOut6 = 0;
16  MachineControl_AnalogOut7 = 0;
17  MachineControl_AnalogOut8 = 0;
18  MachineControl_AnalogOut9 = 0;
19  MachineControl_AnalogOut10 = 0;
20 }
21
22 void loop() {
23 }
```

Digital Inputs

The Portenta Machine Control has up to eight digital input channels, as shown in the image below. Each channel incorporates a voltage divider comprising a 680 k Ω and a 100 k Ω resistor, which scales an input from 0 to 24 VDC down to 0 to 3 VDC.



Portenta Machine Control
digital input channels

Below is an example sketch showcasing how to periodically read data from all the digital input channels.

```
1  /*
2   Portenta Machine Control
3   Name: portenta_machine_control
4   Purpose: This sketch controls the digital inputs
5   from all the digital input channels.
6
7   Author: Arduino PRO Company
8   Version: 1.0 01/10/23
9  */
10
11 #include <Arduino_Portenta_MachineControl.h>
12
13 const int totalChannels = 3;
14 const int channelPins[] = {2, 3, 4};
15
16 void setup() {
17   // Initialize serial communication
18   Serial.begin(9600);
19
20   // Initialize Wire transaction library
21   Wire.begin();
22
23   // Initialize the digital input pins
24   // If initialization fails
25   if (!MachineControl_DigitalInputs.begin(channelPins, totalChannels)) {
26     Serial.println("- Failed to initialize digital inputs");
27   }
28 }
```

Note that the example sketch uses a loop to individually read each digital input channel using the

```
MachineControl_DigitalInputs.read()
```

function from the

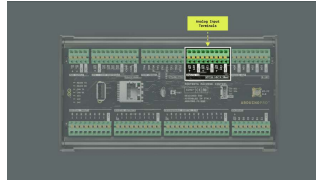
```
Arduino_PortentaMachineControl
```

library. This approach allows for the precise reading of each channel's status in a sequenced manner. The status of each channel is then printed on the IDE's Serial Monitor, ensuring accurate and orderly representation of the digital input states.

Analog Inputs

The Portenta Machine Control has up to three independent analog input

Each channel can have a range resolution varying from 12 to 16 bits, producing decimal values ranging from 0 to 65535, which is configurable through software.



Portenta Machine Control
analog input channels

The configuration of Portenta's Machine Control analog input channels is determined by an analog switch (TS12A44514PWR), which allows switching between three different operational modes:

0-10V: The analog input channel uses a resistor divider consisting of a 100 k Ω and a 39 k Ω resistor for this mode. This scales down an input voltage in the 0 VDC to 10 VDC range to a range of 0 VDC to 2.8 VDC. The resulting input impedance in this configuration is approximately 28 k Ω .

4-20 mA: The analog input channel connects to a 120 Ω resistor for this mode. This configuration allows a 4 mA to 20 mA input current to be converted to a voltage in the 0.48 VDC to 2.4 VDC range.

NTC: For this mode, the input is connected to a 3 VDC voltage reference (REF3330AIRSER). A 100 k Ω resistor is then placed in series, forming a part of the resistor divider powered by the voltage reference.



Each analog input channel has an output voltage pin supplying +24 VDC for powering sensors. This pin has integrated protection through a 500 mA PTC resettable fuse.

To use a specific operational mode with Portenta's Machine Control analog input channels, the

```
MachineControl_AnalogIn.begin(SensorType)
```

function from the

```
Arduino_PortentaMachineControl
```

library must be called before reading values from the analog input channels. Use the following constants in the

```
MachineControl_AnalogIn.begin(SensorType)
```

function to define a specific operational mode:

```
SensorType::V_0_10 : 0-10V mode
```

```
SensorType::MA_4_20 : 4-20mA mode
```

```
SensorType::NTC : NTC mode
```

Below is an example sketch showcasing how to read voltages from the analog input channels set to the 0-10V mode.



```

1  /*
2   Portenta Machine Control
3   Name: portenta_machine_control
4   Purpose: This sketch controls the output of the Portenta Machine Control
5   from the analog input channels of the Portenta Machine Control.
6   of the Portenta Machine Control.
7
8   @author Arduino PRO Company
9   @version 1.0 01/10/23
10 */
11
12 #include <Arduino_Portenta.h>
13
14 // Define the resistor divider constant
15 const float RES_DIVIDER = 0.28;
16
17 // Define the ADC reference voltage
18 const float REFERENCE = 3.0;
19
20 void setup() {
21   // Initialize serial communication
22   Serial.begin(9600);
23
24   // Initialize the analog input channels
25   MachineControl_AnalogIn.begin();
26 }
27
28 void loop() {

```

Note that the example sketch uses the

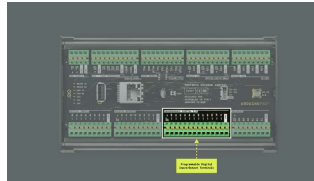
```
MachineControl_AnalogIn.read(channel)
```

function to acquire the raw voltage values from each channel. These raw values are then converted to the actual voltage values using the provided `RES_DIVIDER` and `REFERENCE` constants.

For each analog input channel, there is a resistor divider made by 100K and 39K resistors, meaning that the input voltage is divided by a ratio of 0.28 approximately (`RES_DIVIDER` constant); the analog input channels voltage reference (`REFERENCE` constant) is 3V.

Programmable Digital I/O

The Portenta Machine Control has up to 12 programmable digital input/output channels, as shown in the image below.



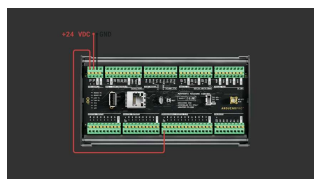
Portenta Machine Control
programmable digital
input/output channels

The programmable digital input/output channels are powered via three quad-channel high-side switches (TPS4H160AQWPRQ1). Each channel comes with a nominal current value of 0.6 A. However, due to internal circuit tolerances of the high-side switches, this value can spike up to 0.9 A.

The programmable digital input/output channels must be connected to an external +24 VDC power supply through pin **24V IN**. This power supply



can be shared with the controller's +24 VDC power supply. Moreover, pin **24V IN** is not galvanically isolated, meaning the input power supply voltage must share the same **GND** as the controller.



Programmable Digital I/O channels power option

There are two modes of overcurrent protection in the programmable digital input/output channels:

- 1 **Latch mode:** The channel is deactivated once the current limit is hit. The respective channel enable pin must be toggled to reactivate the channel.
- 2 **Retry mode:** The channel is momentarily shut down upon reaching the current limit but reconnects briefly.

The programmable digital input/output channels integrate an internal mechanism to protect against kickback from inductive loads. Additionally, there is an external safeguard via a 60 VDC, 2 A Schottky diode (PMEG6020ER). Each of the 12 digital input channels uses a resistor divider setup consisting of a 680 k Ω and 100 k Ω resistor; this configuration scales down a 0 to 24 VDC input to a 0 to 3 VDC range. While the high-side switches function separately from the digital input channels, it is possible to read the status of the high-side switches via the digital input channels.

Ensure each channel does not exceed a maximum current of 500 mA to avoid potential damage or malfunctions in the programmable digital input/output channels.

The sketch below showcases using the programmable digital input/output channels of the Portenta Machine Control. This


```
MachineControl_DigitalPro  
grammables.set(pin, state)
```

: Used to define a particular channel's state (ON/OFF).

```
MachineControl_DigitalPro  
grammables.read(pin)
```

: Used to discern the state of a specific channel.

```
MachineControl_DigitalPro  
grammables.writeAll(state)
```

: Used to configure the state (ON/OFF) for all available pins or channels simultaneously.

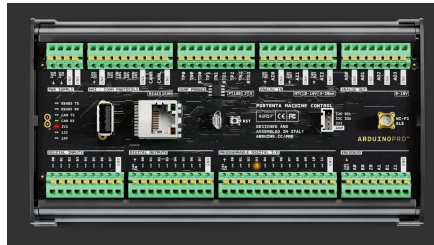
```
MachineControl_DigitalPro  
grammables.readAll()
```

: Used to read the states of all available channels collectively.

```
MachineControl_DigitalPro  
grammables.toggle()
```

: Used to invert the states of all the channels.

Open the Serial Monitor to watch the I/Os states. The sketch will showcase the reading and state control process of the I/Os of the channels.



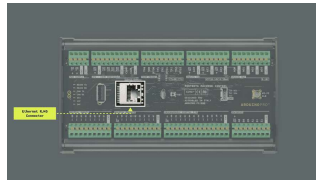
Communication

This user manual section covers the different communication interfaces and protocols the Portenta Machine Control supports, including the [Ethernet](#), [Wi-Fi®](#), [Bluetooth®](#), [RS-485](#), [Modbus](#), and [CAN Bus](#).

Ethernet

The Portenta Machine Control features an onboard low-power

transceiver complies with the IEEE 802.3 and 802.3u standards and supports communication with an Ethernet MAC through a standard RMII interface. The Ethernet transceiver is accessible through the onboard RJ45 connector.



Onboard RJ45 connector of
the Portenta Machine
Control

The

Arduino Mbed OS Portenta Boards

core has a built-in library that lets you use the onboard Ethernet PHY transceiver right out of the box: the [Ethernet library](#). Let's use an example sketch demonstrating some of the transceiver's capabilities.

The sketch below enables a Portenta Machine Control to connect to the Internet via an Ethernet connection. Once connected, it performs a `GET` request to the `ip-api.com` service to fetch details about the device's IP address. It then parses the received JSON object using the `Arduino_JSON` [library](#) to extract key IP details: IP address, city, region, and country. This data is then printed to the Arduino IDE's Serial Monitor.

```
1  /**
2   Portenta's Machine Code
3   Name: portenta_machine_code
4   Purpose: This sketch
5   to ip-api.com via Ethernet
6   the controller
7
8   @author Arduino PRO Core Team
9   @version 1.0 01/10/2023
10 */
11
12 // Include the necessary libraries
13 #include <Ethernet.h>
14 #include <Arduino_JSON.h>
15
16 // Server address for ip-api.com
17 const char* server = "ip-api.com";
18
19 // API endpoint path to get IP details
20 String path = "/json/";
21
22 // Static IP configuration
23 // Used only if DHCP is not available
24 IPAddress ip(10, 130, 2, 1);
25
26 // Ethernet client instance
27 EthernetClient client;
28
```

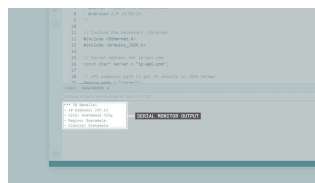
The sketch includes the `Ethernet` and `Arduino_JSON` libraries, which are essential for Ethernet and JSON handling functionality. In the `setup()` function, serial communication is initiated for debugging and output. Instead of DHCP, the Ethernet connection uses a predefined static IP address.

Once the Ethernet connection runs, the sketch connects to the `ip-api.com` service, utilizing the HTTP protocol. Specifically, an `HTTP GET` request is crafted to retrieve details about the device's IP address, including its city, region, and country. If the connection to the server fails, the sketch will output an error message to the Arduino IDE's

Within the `loop()` function, an HTTP GET request is sent to the `ip-api.com` service once. The sketch then waits for and skips the response's HTTP headers, parsing the following JSON payload.

Key IP details such as IP address, city, region, and country are extracted and then displayed in the IDE's Serial Monitor using the parsed data. If the Ethernet link happens to be disconnected, a corresponding message is printed to the Serial Monitor. Should the JSON parsing fail, an error message is showcased on the IDE's Serial Monitor, prompting the sketch to exit the current iteration of the `loop()` function immediately.

You should see the following output in the Arduino IDE's Serial Monitor:



Example sketch output in the Arduino IDE's Serial Monitor

Wi-Fi®

The Portenta Machine Control features an onboard Wi-Fi® module that provides seamless wireless connectivity, allowing it to connect to Wi-Fi® networks and interact with other devices Over-The-Air (OTA).



Onboard SMA female antenna connector of the Portenta Machine Control

Some of the key capabilities of Portenta's Machine Control onboard Wi-Fi® module are the following:

Wireless connectivity: The onboard Wi-Fi® module supports IEEE 802.11b/g/n Wi-Fi® standards, enabling devices to establish reliable and high-speed wireless connections to access the Internet and communicate with other devices.

Secure communication: The onboard module incorporates various security protocols such as WEP, WPA, WPA2, and WPA3, ensuring robust data encryption and protection against unauthorized access during wireless communication.

Antenna connector: Portenta Machine Control devices feature an onboard vertical SMA female antenna connector (5-1814832-2) specifically matched for the onboard Wi-Fi® module RF requirements. For external antenna testing, use a standard SMA male antenna.

The

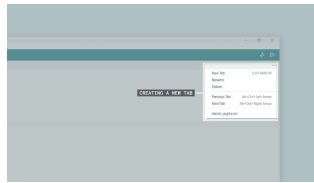
Arduino Mbed OS Portenta Boards

core has a built-in library that lets you use the onboard Wi-Fi® module right out of the box: the [WiFi library](#). Let's walk through an example sketch demonstrating some of the module's capabilities.

The sketch below enables a Portenta Machine Control device to connect to the Internet via Wi-Fi® (like the Ethernet example). Once connected, it performs a `GET` request to the `in-ani.com` server to fetch details

parses the received JSON object using the `Arduino_JSON` library to extract key IP details: IP address, city, region, and country. This data is then printed to the Arduino IDE's Serial Monitor.

You need to create first a header file named `arduino_secrets.h` to store your Wi-Fi® network credentials. To do this, add a new tab by clicking the ellipsis (the three horizontal dots) button on the top right of the Arduino IDE 2.



Creating a tab in the
Arduino IDE 2

Put `arduino_secrets.h` as the "Name for new file" and enter the following code on the header file:

```
1 #define SECRET_SSID "YOUR_  
2 #define SECRET_PASS "YOUR_
```

Replace `YOUR_SSID` with the name of your Wi-Fi® network and `YOUR_PASS` with the password of it and save the project. The example sketch is as follows:

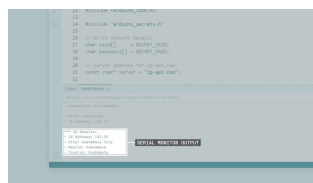
```
1  /**
2   Portenta's Machine Co
3   Name: portenta_machir
4   Purpose: This sketch
5   to ip-api.com via Wi-
6
7   @author Arduino PRO C
8   @version 1.0 01/10/23
9  */
10
11  #include <WiFi.h>
12  #include <Arduino_JSON.
13
14  // Wi-Fi network detail
15  char ssid[] = SECRET_SS
16  char password[] = SECRE
17
18  // Server address for i
19  const char* server = "i
20
21  // API endpoint path to
22  String path = "/json";
23
24  // Wi-Fi client instanc
25  WiFiClient client;
26
27  // JSON variable to sto
28  JSONVar doc;
```

The sketch includes the `WiFi` and `Arduino_JSON`, which provide the necessary Wi-Fi® and JSON handling functionality. The `setup()` function initiates serial communication for debugging purposes and attempts to connect to a specified Wi-Fi® network. If the connection is not established, the sketch will keep trying until a successful connection is made.

Once the Wi-Fi® connection is established, the sketch is ready to connect to the `ip-api.com` server using the HTTP protocol. Specifically, an `HTTP GET` request is constructed to query details related to its IP address. The `GET` request is sent only once after the Wi-Fi® connection

The `loop()` function is the heart of the sketch. It checks whether the data has been fetched or not. If the data still needs to be fetched, it tries to establish a connection to the server. If the connection is successful, the sketch sends an `HTTP GET` request, skips the HTTP headers of the response, and uses the `JSON.parse()` function from the `Arduino_JSON` library to parse the JSON object from the response. The parsed data extracts key IP details like IP address, city, region, and country, which are then printed to the Arduino IDE's Serial Monitor. Once the data is published, the client is disconnected to free up resources. Suppose the JSON parsing fails for any reason. In that case, an error message is outputted to the Arduino IDE's Serial Monitor, and the sketch immediately exits the current iteration of the `loop()` function.

Since the data is fetched only once, there's no need to send `HTTP GET` requests repeatedly. After the initial fetch, you should see the details related to the IP address displayed in the Arduino IDE's Serial Monitor:

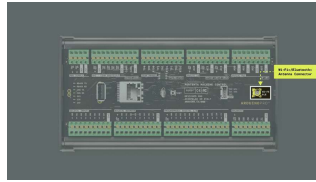


Example sketch output in the Arduino IDE's Serial Monitor

Bluetooth®

The Portenta Machine Control features an onboard Bluetooth®

connect to other Bluetooth® devices and networks.



Onboard SMA female antenna connector of the Portenta Machine Control

To enable Bluetooth® communication on the Portenta Machine Control, you can use the [ArduinoBLE library](#). Let's use an example code demonstrating some of the capabilities of Poternta's Machine Control Bluetooth® module. Here is an example of how to use the `ArduinoBLE` library to create a temperature monitor application:



```
1  /**
2   Portenta's Machine Control
3   Name: portenta_machine_control
4   Purpose: Read temperature from
5   the Portenta Machine Control
6   value using a Bluetooth Low
7
8   @author Arduino PRO Company
9   @version 1.0 01/10/23
10 */
11
12 #include <ArduinoBLE.h>
13 #include <Arduino_Portenta.h>
14
15
16 // Define the environment
17 BLEService temperatureService("1915");
18 BLEIntCharacteristic temperatureCharacteristic(1);
19
20
21 void setup() {
22   // Initialize the digital pins
23   MachineControl_Digital_Pins_Init();
24
25   // Initialize serial communication
26   Serial.begin(9600);
27
28   // Initialize the Portenta Machine Control
```

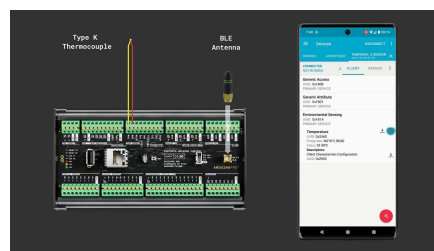
The example sketch shown above uses a standard Bluetooth® Low Energy service and characteristic for transmitting temperature read by one of the thermocouple input terminals of the Portenta Machine Control to a central device.

The sketch begins by importing all the necessary libraries and defining the Bluetooth® Low Energy service and characteristics.

In the `setup()` function, the code initializes the Portenta Machine Control and sets up the Bluetooth® Low Energy service and characteristics. Then, it begins advertising the defined Bluetooth® Low Energy service.

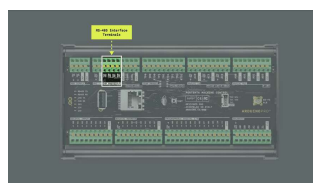
A Bluetooth® Low Energy

connects to the Portenta Machine Control, channel 0 of its digital output terminals is turned on. The sketch then enters into a loop that constantly reads the temperature from a thermocouple input terminal. The temperature is printed to the IDE's Serial Monitor and transmitted to the central device over the defined Bluetooth® Low Energy characteristic.



RS-485 (Half/Full Duplex)

The Portenta Machine Control has a built-in RS-485 interface that enables the implementation of robust and reliable data transmission systems. RS-485 interface is a protocol widely used in industrial applications. The wide common-mode range enables data transmission over longer cable lengths and in noisy environments such as the floor of a factory. Also, the high input impedance of the receivers allows more devices to be attached to the communication lines.



Portenta Machine Control
RS-485 interface terminals

The onboard RS-485 transceiver is the SP325 from Maxim. The

transceiver that supports RS-232, RS-485, and RS-422 serial standards.

Integrated cable termination and multiple configuration modes allow all three protocols to be used interchangeably over a single cable or connector with no additional switching components.

 The Portenta Machine Control has onboard termination resistors; its RS-485 interface can be configured to be half-duplex or full-duplex.

Some of the key capabilities of Portenta's Machine Control onboard RS-485 transceiver are the following:

High-speed operation: The RS-485 transceiver can operate up to 20 Mbps.

EMI Reduction: The slew rate is limited to 250 kbps to minimize electromagnetic interference (EMI), enhancing signal integrity.

ESD protection: Transmitter outputs and receiver inputs are protected against electrostatic discharge (ESD) up to ± 15 kV.

High impedance: The transceiver inputs exhibit high impedance, allowing up to 256 devices on a single communication bus.

Communication mode: The transceiver can be configured either half or full-duplex.

Termination resistors: 120 Ω termination resistors are integrated and can be connected or disconnected through software.

RS-485 data lines in the Portenta Machine Control are labeled as described in the following table:

Pin Name	RS-485 Full-duplex	RS-485 Half-duplex
RS485 TX P	TX+	Data+
RS485 TX N	TX-	Data-
RS485 RX P	RX+	-
RS485 RX N	RX-	-

RS-485 data line labels differ between manufacturers. Most manufacturers will use **+** and **-** to label the data lines or variations such as **D+** and **D-**. Some manufacturers will label inputs as **A** and **B** but get the polarity backward, so **A** is positive and **B** negative.



Although predicting how other manufacturers will mark these lines is impossible, practical experience suggests that the **-** line should be connected to the **A** terminal. The **+** line should be connected to the **B** terminal. Reversing the polarity will not damage an RS-485 device, but the communication will not work as expected.

The example sketch below shows how to use the RS-485 interface of the Portenta Machine Control for half-duplex communication.

```

1  /*
2   Portenta Machine Control
3   Name: portenta_machine_control
4   Purpose: Demonstrates RS-485
5   the Portenta Machine Control
6
7   @author Arduino PRO Core Team
8   @version 1.0 01/10/23
9  */
10
11 // Include the necessary libraries
12 #include <Arduino_PortentaMachineControl.h>
13
14 // Define the interval between transmissions
15 constexpr unsigned long interval = 1000;
16 unsigned long sendNow = 0;
17 unsigned long counter = 0;
18
19 void setup() {
20     // Initialize serial
21     Serial.begin(9600);
22
23     // Wait for the serial port to be ready
24     // This is necessary for the RS-485 module
25     while (!Serial);
26
27     Serial.println("- Initial setup complete");
28
29     // Initialize the RS-485 module
30     MachineControl_RS485Comm.begin(9600, pre, post);
31 }
32
33 void loop() {
34     // Send a message over RS-485
35     MachineControl_RS485Comm.send("Hello, World!");
36     counter++;
37     sendNow = millis();
38 }

```

In this example sketch, a message is periodically sent over the RS-485 interface of the Portenta Machine Control. The sketch initializes the RS-485 interface for half-duplex communication and sends a `String` message with a counter. After each transmission, it switches back to receive mode to listen for incoming data.

The example sketch uses the following functions from the `Arduino_PortentaMachineControl` library for RS-485 communication. Here is an explanation of the functions:

`MachineControl_RS485Comm.begin(baud, pre, post)`
: Initializes the RS-485 module

```
MachineControl_RS485Comm.receive()
```

: Puts the module in receive mode.

```
MachineControl_RS485Comm.noReceive()
```

: Disables receive mode for transmission.

```
MachineControl_RS485Comm.beginTransmission()
```

: Prepares the module to start transmitting data.

```
MachineControl_RS485Comm.endTransmission()
```

: Ends data transmission and prepares the module to receive data.

```
MachineControl_RS485Comm.available()
```

: Checks if data can be read.

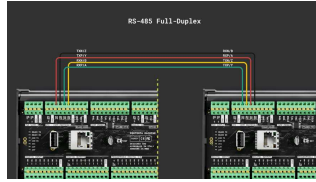
```
MachineControl_RS485Comm.read()
```

: Reads incoming data.

To receive and show the messages on your computer, you can use a USB to RS-485 converter, such as [the converter used by the Arduino Pro Content Team](#). You can use the Arduino IDE's Serial Monitor to display the messages

 received in the converter or another serial terminal such as [CoolTerm](#), a simple and cross-platform (Windows, Mac, and Linux) serial port terminal application (no terminal emulation) that is geared towards hobbyists and professionals.

As a practical example, we will **establish a full duplex communication between two Portenta Machine Control devices**. Follow the wiring below for the RS-485 full-duplex communication.



Full-duplex RS-485 wiring

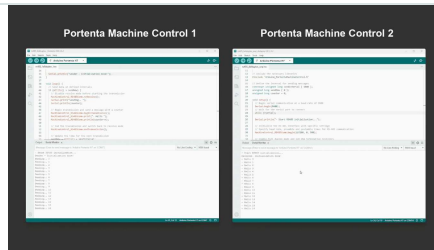
For both **Portenta Machine Control** devices, use the example sketch shown below; this example sketch can also be found on the Arduino IDE by navigating to **File > Examples > Arduino_PortentaMachineControl > RS485_fullduplex**.

```

1  /*
2   * Portenta Machine Control
3   * Name: RS485_fullduplex
4   * Purpose: Demonstrates
5   * the Portenta Machine
6   * and receive data per
7   *
8   * @author Riccardo Rizzo
9   * @version 1.0 01/10/20
10  */
11
12  // Include the necessary
13  #include "Arduino_PortentaMachineControl.h"
14
15  // Define the interval t
16  constexpr unsigned long
17  unsigned long sendNow {
18  unsigned long counter =
19
20  void setup() {
21    // Initialize serial c
22    // Wait for the serial
23    Serial.begin(9600);
24    while (!Serial);
25
26    Serial.println("- Star
27
28    // Initialize the RS-4

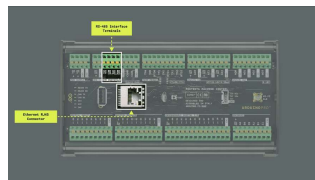
```

Both devices will send and receive messages respectively through the RS-485 interface and will print them



Modbus (RTU/TCP)

The Portenta Machine Control incorporates a built-in Modbus interface, enabling the implementation of robust and reliable data transmission systems. Modbus, in its RTU version that operates RS-485 serial transmission or in its TCP version that works over Ethernet, remains one of the most widely used protocols for industrial automation applications, building management systems, and process control, among others.



Portenta Machine Control
RS-485 interface terminals
and onboard RJ45 Ethernet
connector

The many nodes connected in a Modbus network, whether RTU or TCP, allow high flexibility and scalability in constructing automation and control systems.

To use the Modbus protocol with your Portenta Machine Control, you will need the `ArduinoRS485` and `ArduinoModbus` libraries. You can install them via the Library Manager of the Arduino IDE.

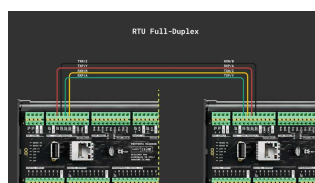
Modbus RTU, generally operating in half-duplex mode, has the capability to handle noisy and long-distance transmission lines, which makes it an excellent choice for industrial environments. Modbus RTU communication is supported using Portenta Machine Control's RS-485 physical interface.

The Portenta Machine Control has onboard termination resistors; its RS-485 interface can be configured as a half or full duplex.

Using Two Portenta Machine Controls

The following example shows how to establish Modbus RTU communication between **two Portenta Machine Control devices**. We will use four LEDs from the Digital Output port of the Portenta Machine Control as the visual indicator to confirm the communication is working as intended. Since Portenta Machine Control supports half-duplex and full-duplex modes, each mode requires a different wiring setup.

We will begin showing the **Full-Duplex mode** example following the connection diagram below between two Portenta Machine Control devices:



Modbus RTU (Full-Duplex)

The following script assigns a Portenta Machine Control as a Client device, which sends four coil values to the Portenta Machine Control assigned as a Server.

```

1 // Include the necessary
2 #include <Arduino_Portenta.h>
3 #include <ArduinoRS485.h>
4 #include <ArduinoModbus.h>
5
6 // Define the baud rate
7 constexpr auto baudrate = 9600;
8
9 // Calculate preDelay and postDelay
10 // Modbus over serial line
11 // Paragraph 2.5.1.1 Modbus
12 // https://modbus.org/docs/Modbus_Protocol_Specification_V1_02.pdf
13 constexpr auto bitduration = 1000000 / baudrate;
14 constexpr auto preDelay = 10 * bitduration;
15 constexpr auto postDelay = 10 * bitduration;
16
17 // Counter variable to count
18 int counter = 0;
19
20 void setup() {
21     // Begin serial communication
22     Serial.begin(baudrate);
23
24     // Wait for serial port to be ready
25     while (!Serial);
26     // Initialize RS-485 communication
27     MachineControl_RS485Comm.begin(baudrate);
28
29     // Initialize Modbus communication
30     MachineControl_RS485Comm.begin(baudrate, preDelay, postDelay);
31 }
32
33 void loop() {
34     // Send four coil values to the server
35     MachineControl_RS485Comm.writeCoil(0, true);
36     MachineControl_RS485Comm.writeCoil(1, false);
37     MachineControl_RS485Comm.writeCoil(2, true);
38     MachineControl_RS485Comm.writeCoil(3, false);
39
40     // Increment counter
41     counter++;
42
43     // Delay between writes
44     delay(1000);
45 }

```

Because the Portenta Machine Control is operating in **Full-Duplex mode**, the following line of code is essential and must be included within the code to enable Full-Duplex mode:

```

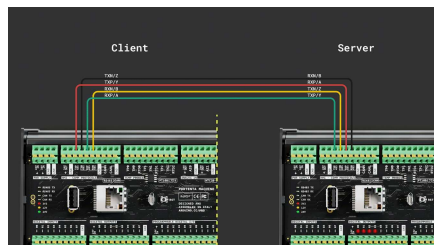
1 MachineControl_RS485Comm.setFullDuplex(true);

```

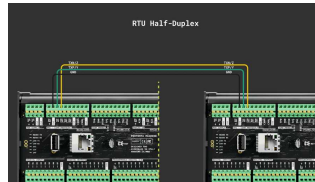
The Server Portenta Machine Control uses the script below, which translates received coil values into corresponding Digital Outputs. It will blink four Digital Outputs accordingly in a timely manner.

```
1 // Include the necessary
2 #include <ArduinoRS485.h>
3 #include <ArduinoModbus.h>
4 #include <Arduino_Portenta.h>
5
6 // Define the number of
7 const int numCoils = 4;
8
9 // Define the baud rate
10 constexpr auto baudrate = 115200;
11
12 // Calculate preDelay and postDelay for
13 // Modbus over serial line
14 // Paragraph 2.5.1.1 Modbus Specification
15 // https://modbus.org/docs/Modbus_Specification.pdf
16 constexpr auto bitduration = 1000000 / baudrate;
17 constexpr auto preDelay = 10 * bitduration;
18 constexpr auto postDelay = 10 * bitduration;
19
20 void setup() {
21     // Begin serial communication
22     Serial.begin(9600);
23
24     // Print a startup message
25     Serial.println("- Modbus Server");
26
27     // Set RS485 transmission mode
28     MachineControl_RS485Control.setMode(MachineControl_RS485Control::MODE_SERVER);
```

With this, we have two Portenta Machine Control devices, each assigned as a Client and Server correspondingly, communicating with Modbus RTU in full-duplex mode.



Alternatively, to establish communication between two Portenta Machine Control with Modbus RTU in **Half-Duplex mode**, the following wiring setup is required:

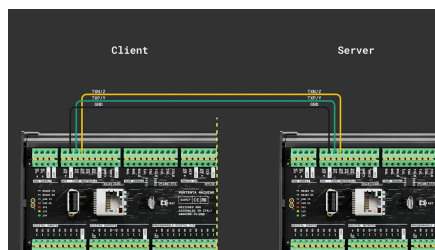


Modbus RTU (Half-Duplex)
between two Portenta
Machine Control

The previous example can be used in half-duplex mode, but it will require one minor change in the following line of code:

```
1 MachineControl_RS485Comm.s
```

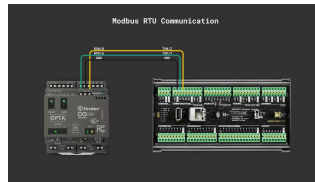
By modifying this line, *Full-Duplex mode* is deactivated. This minor code tweak, plus the appropriate wiring, enable the two units to communicate using Modbus RTU in half-duplex mode.



Using a Portenta Machine Control & Opta™

The example below shows how to enable Modbus RTU communication between a Portenta Machine Control

wiring both devices, follow the diagram below:



Portenta Machine Control
and Opta™ Modbus RTU
wiring

The following example sketch will let the Portenta Machine Control device toggle the four onboard Opta™ LEDs via **Modbus RTU**. The Portenta Machine Control will be the **client**, while the Opta™ device will be the **server**.

For the **Opta™** device, defined as the server, use the following example sketch below.

```
1  /*
2   Opta's Modbus RTU Serv
3   Name: opta_modbus_rtu.
4   Purpose: Demonstrates
5   Opta device using the
6
7   @author Arduino PRO Co
8   @version 1.0 01/10/23
9  */
10
11 // Include the necessary
12 #include <ArduinoRS485.h>
13 #include <ArduinoModbus.h>
14
15 // Define the baud rate
16 constexpr auto baudrate = 115200;
17
18 // Define the number of
19 const int numCoils = 4;
20
21 // Calculate preDelay ar
22 // Modbus over serial li
23 // Paragraph 2.5.1.1 Moc
24 // https://modbus.org/dc
25 constexpr auto bitdurati
26 constexpr auto preDelayE
27 constexpr auto postDelay
28
```

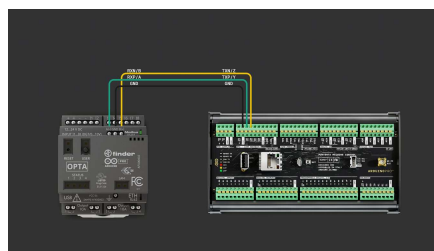
For the **Portenta Machine Control**,
defined as the client, use the
following example sketch.


```

1  /*
2   Portenta's Machine Core
3   Name: portenta_machine
4   Purpose: Demonstrates
5   Portenta Machine Control
6
7   @author Arduino PRO Core
8   @version 1.0 01/10/23
9  */
10
11 // Include the necessary
12 #include <Arduino_Portenta.h>
13 #include <ArduinoRS485.h>
14 #include <ArduinoModbus.h>
15
16 // Define the baud rate
17 constexpr auto baudrate = 115200;
18
19 // Calculate preDelay and postDelay for
20 // Modbus over serial line
21 // Paragraph 2.5.1.1 Modbus
22 // https://modbus.org/docs/Modbus_Specification_V1_02.pdf
23 constexpr auto bitduration = 1000000 / baudrate;
24 constexpr auto preDelay = 10 * bitduration;
25 constexpr auto postDelay = 10 * bitduration;
26
27 // Counter variable to count
28 int counter = 0;

```

You should see the four onboard LEDs of the Opta™ device turn on and off, as shown below.



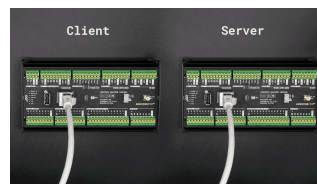
Modbus TCP

Modbus TCP, taking advantage of Ethernet connectivity, allows easy integration with existing computer networks and facilitates data communication over long distances using the existing network infrastructure. It operates in full-

The example below shows how to enable Modbus TCP communication between two Portenta Machine Controls. For wiring both devices, we have two options:

- 1 A direct connection between the Portenta Machine Controls through an Ethernet cable.
- 2 Individually connect each device to an internet router via Ethernet cables.

We will use the second option since it will allow us to scale the application by adding more devices to the network.



Modbus TCP Ethernet
Wiring

The following example sketch will let one Portenta Machine Control toggle the digital output LEDs of a second Portenta Machine Control through the Modbus TCP protocol. One Portenta Machine Control will be the **client**, and the other will be the **server**; as they are both connected to an internet router, IP addresses are the way for them to route their messages.

We can define a **Static** or **DHCP** IP address to them using the function

`Ethernet.begin()` as follows:

```

1 // DHCP (will assign an IP
2 Ethernet.begin();
3
4 // Static IP
5 Ethernet.begin(<mac>, <IP>

```



The client must know the server IP to establish communication between them.

Use the following example sketch for the **Portenta Machine Control** defined as the client.

```

1 /*
2  Portenta Machine Control
3  Name: portenta_machine
4  Purpose: Demonstrates
5  Modbus TCP protocol on
6
7  @author Arduino PRO Co
8  @version 1.0 01/10/23
9  */
10
11 // Include necessary libraries
12 #include <SPI.h>
13 #include <Ethernet.h>
14 #include <ArduinoRS485.h>
15 #include <ArduinoModbus.h>
16
17 EthernetClient ethClient;
18 ModbusTCPClient modbusTCP;
19
20 // Define the IP address
21 IPAddress ip(10, 0, 0, 1);
22
23 // Define the IP Address
24 IPAddress server(10, 0, 0, 2);
25
26 void setup() {
27   // Initialize serial
28   // wait for the serial
29   Serial.begin(9600);
30 }

```

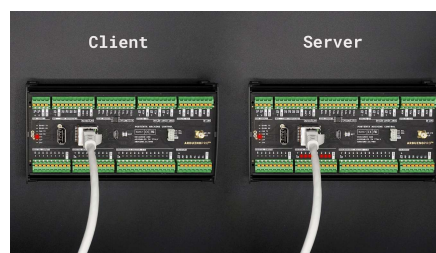
For the second **Portenta Machine Control** defined as the server, use the following example sketch.

```

1  /*
2   Portenta Machine Control
3   Name: pmc_modbus_tcp_server
4   Purpose: Demonstrates
5   Opta device to control
6
7   @author Arduino PRO Co
8   @version 1.0 01/10/23
9  */
10
11  #include <SPI.h>
12  #include <Ethernet.h>
13  #include <ArduinoRS485.h>
14  #include <ArduinoModbus.h>
15  #include <Arduino_Portenta.h>
16
17  // Define the IP address
18  IPAddress ip(10, 0, 0, 2
19
20  // Server will listen on
21  EthernetServer ethServer
22
23  // Create a Modbus TCP s
24  ModbusTCPServer modbusTC
25
26  void setup() {
27    // Initialize serial c
28    // wait for the serial

```

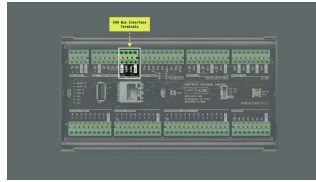
You should see the server digital output LEDs turning on and off, as shown below:



CAN Bus

The Portenta Machine Control features a built-in CAN bus interface, enabling the implementation of robust and reliable data transmission systems in automotive and industrial automation applications. The CAN bus is widely used due to its ability to

communication method that reduces errors.



Portenta Machine Control
CAN bus interface
terminals

The onboard CAN transceiver of the Portenta Machine Control is the TJA1049 from NXP®. The TJA1049 is a specialized high-speed CAN transceiver for various applications, especially in automotive and high-speed CAN networks. The third-generation device offers enhanced electromagnetic compatibility (EMC) and ESD protection. This transceiver also features a low-current standby mode with a wake-up function and is compatible with microcontrollers ranging from 3 to 5 VDC. Adhering to the ISO11898 standard, the TJA1049 ensures reliable communication at data rates up to 5 Mbps, making it an optimal choice for High-Speed (HS) CAN networks that require efficient low-power operation modes.

Some of the key capabilities of the onboard CAN transceiver in the Portenta Machine Control include:

High-speed operation: The onboard transceiver can operate at bit rates up to 5 Mbps.

Noise tolerance: The onboard transceiver is designed to function reliably in environments with high electromagnetic interference.

Low-current standby mode

mode, which includes efficient wake-up capabilities, crucial for energy-efficient applications.

Compliance with ISO11898

standard: Adhering to the ISO11898 standard, the TJA1049 ensures reliable communication at data rates up to 5 Mbit/s, making it ideal for HS CAN networks operating in low-power modes.

The example sketch below shows how to use the CAN bus interface of the Portenta Machine Control to transmit data. You can also find it in

File > Examples >

Arduino_PortentaMachineControl > CAN > WriteCan.

```

1  /*
2   Portenta Machine Control
3   Name: WriteCan.ino
4   Purpose: Demonstrates
5   interface on the Portenta
6
7   @author Arduino PRO Core Team
8   @version 1.0 01/10/23
9  */
10
11 // Include necessary libraries
12 #include <Arduino_PortentaMachineControl.h>
13
14 // Define the CAN message
15 static uint32_t const CAN_ID = 0x12345678;
16 static uint32_t msg_cnt = 0;
17
18 void setup() {
19     // Initialize serial
20     Serial.begin(9600);
21
22     // Wait 2.5 seconds
23     for (const auto time : {2500}) {
24         delay(time);
25     }
26     // Initialize the CAN
27     if (!MachineControl.canInit()) {
28         Serial.println("CAN init failed");
29         while(1);
30     }
31 }
32
33 void loop() {
34     // Transmit a message
35     MachineControl.canWrite(CAN_ID, "Hello CAN!");
36     msg_cnt++;
37     if (msg_cnt == 10) {
38         Serial.println("10 messages transmitted");
39         msg_cnt = 0;
40     }
41 }

```

The example sketch uses the

library for CAN communication. Here is an explanation of these functions:

```
MachineControl_CANComm.begin(bitRate)
```

: Initializes the CAN module with a specified bit rate.

```
MachineControl_CANComm.write(msg)
```

: Transmits a data message over the CAN network. The `msg` parameter contains the data to be sent.

```
MachineControl_CANComm.available()
```

: Checks if data is available on the CAN bus to be read.

```
MachineControl_CANComm.read()
```

: Reads incoming data from the CAN bus. This function is used to retrieve data that has been received.

```
MachineControl_CANComm.end()
```

: This function can disable the CAN module when it's no longer needed, helping conserve power.

The example sketch below shows how to use the CAN bus interface of the Portenta Machine Control to read data. You can also find it in **File >**

Examples >

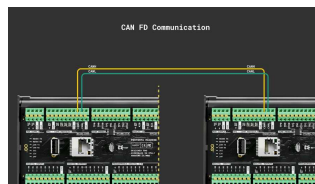
Arduino_PortentaMachineControl > CAN > ReadCan.

```

1  #include <Arduino_Portenta.h>
2
3  void setup() {
4    // Initialize serial communication
5    // wait for serial port to open
6    Serial.begin(9600);
7    while (!Serial) {
8      ;
9    }
10
11    if (!MachineControl_CAN) {
12      Serial.println("- CAN not initialized");
13      while(1) ;
14    }
15  }
16
17  void loop() {
18    if (MachineControl_CAN) {
19      CanMsg const msg = MachineControl_CAN;
20      Serial.println(msg);
21    }
22  }

```

As a practical example, we will establish a CAN communication between two Portenta Machine Control devices. Follow the wiring shown below for the CAN communication.



CAN communication wiring

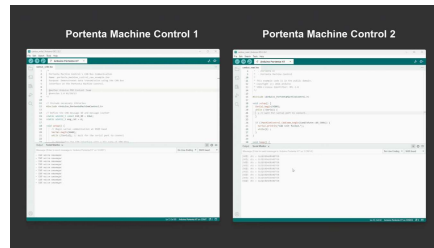


The Portenta Machine Control has built-in 120 Ω termination resistors.

For one of the devices, use the [CAN writing example](#); for the other one, use the [CAN reading example](#) explained above.

The Portenta Machine Control will

CAN bus; you can see the received message using the Arduino's IDE Serial Monitor.



To receive and show the messages on your computer, you can use a USB to CAN bus converter as an [example](#). You can use the Arduino IDE's Serial Monitor to display the messages received in the

i converter or another serial terminal such as [CoolTerm](#), a simple and cross-platform (Windows, Mac, and Linux) serial port terminal application (no terminal emulation) that is geared towards hobbyists and professionals.

Real-Time Clock

The Portenta Machine Control features an onboard CMOS Real-Time Clock (RTC) and calendar, the PCF8563 from NXP®, optimized for low power consumption.

Some of the key capabilities of Portenta's Machine Control onboard RTC are the following:

Timekeeping accuracy:

Provides year, month, day, weekday, hours, minutes, and seconds based on a 32.768 kHz quartz crystal.

Alarm and timer functions:

Offers additional utility for time-based alerts and operations

timekeeping reliability and stability.

Internal Power-On Reset

(POR): Ensures consistent performance and reliable operation.

Open-drain interrupt pin:

Facilitates external notifications and system wake-up.

The

Arduino Mbed OS Portenta Boards

core and the

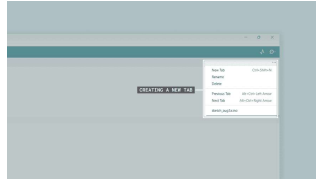
Arduino_PortentaMachineControl

are equipped with built-in libraries and functions that enable you to utilize the Portenta's Machine Control onboard Real-Time Clock (RTC), connect to Wi-Fi® networks and work with time functions using the `mbed_mktime` library. In the following example, we will explore some of these capabilities.

The following example sketch demonstrates how to connect a Portenta Machine Control device to a Wi-Fi® network, synchronize its onboard RTC with a Network Time Protocol (NTP) server using the `NTPClient` library, and display the current RTC time on the Arduino IDE's Serial Monitor every five seconds. To get started, you will need to install the `NTPClient` library, which can be easily added using the Arduino IDE's Library Manager.

Before running the sketch, create a header file named

`arduino_secrets.h` to securely store your Wi-Fi network credentials. In the Arduino IDE 2, this can be done by adding a new tab. Click the ellipsis (the three horizontal dots) button at the top right of the IDE, and name the new file `arduino_secrets.h`.



Creating a tab in the
Arduino IDE 2

In this file, define your Wi-Fi network
SSID and password as constants.

```
1 char ssid[] = "SECRET_SSID";  
2 char password[] = "SECRET_PASS";
```

Replace `SECRET_SSID` with the name
of your Wi-Fi® network and
`SECRET_PASS` with the password of it
and save the project. The example
sketch is as follows:

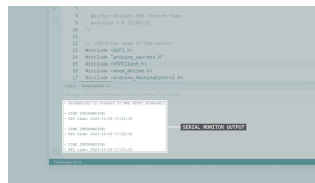


```
1  /*
2   Portenta Machine Contr
3   Name: portenta_machin
4   Purpose: Connects the
5   and synchronizes its c
6   the current RTC time c
7
8   @author Arduino PRO Co
9   @version 1.0 23/07/23
10 */
11
12 // Libraries used in the
13 #include <WiFi.h>
14 #include "arduino_secre
15 #include <NTPClient.h>
16 #include <mbed_mktime.h>
17 #include <Arduino_Porte
18
19 // Wi-Fi network credent
20 int status = WL_IDLE_STA
21
22 // NTP client configurat
23 WiFiUDP ntpUDP;
24 NTPClient timeClient(ntp
25
26 // Display time every 5
27 unsigned long interval =
28 unsigned long lastTime =
```

This sketch uses `WiFi.h`, `NTPClient.h`, and `mbed_mktime.h` libraries and methods to connect to a specific Wi-Fi® network using the provided credentials (network name and password). Once the internet connection has been established, the sketch synchronizes with an NTP server, using the `NTPClient.h` library, to obtain the current Coordinated Universal Time (UTC). This time is then converted to local time and used to set the device's internal RTC, thanks to the functionalities provided by `mbed_mktime.h` methods.

Once the RTC has been synchronized in the setup, the sketch enters an infinite loop. In this loop, every five

Serial Monitor in a more readable format, using the `tm` structure provided by `MBED_MKTIME_H`. This ensures that even if the internet connection is interrupted or the system restarts, accurate time tracking is maintained as long as the RTC's power supply is not interrupted. You should see the following output in the Arduino IDE's Serial Monitor:



Example sketch output in the Arduino IDE's Serial Monitor

The sketch uses several key functions and methods:

`WiFi.begin(ssid, password)`:

Connects the device to a specified Wi-Fi network.

`NTPClient`: A client object to communicate with an NTP server.

`MachineControl_RTCController.begin()`

: Initializes the onboard RTC.

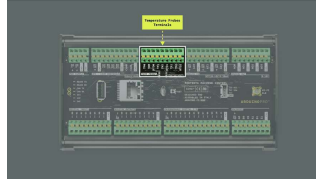
`MachineControl_RTCController.setEpoch(epoch)`

: Sets the RTC time based on the epoch time obtained from the NTP server.

`displayRTC()`: A custom function to format and display the current time from the RTC on the IDE's Serial Monitor.

Temperature Measurements

temperature measurement channels, enhancing its capabilities for various industrial and environmental monitoring applications.



Portenta Machine Control
temperature probes
terminals

The Portenta Machine Control is compatible with the following temperature probes:

Type K Thermocouples

Type J Thermocouples

PT100 probes

Connect only non-grounded thermocouples. For more information about how to connect thermocouples to the Portenta Machine Control, please refer to the [complete pinout available here](#).

The Portenta Machine Control includes two onboard specialized front ends:

MAX31855: This front end is dedicated to **thermocouples**, providing accurate temperature measurements for a wide range of thermocouple types.

MAX31865: This front end is dedicated to **PT100 sensors**, ensuring precise temperature readings for these sensors.

The two onboard front ends are connected to the three temperature channels through a multiplexing system based on the following analog

NX3L4053: A single low-ohmic single-pole double-throw switch that selects between the two front ends.

TMUX1511: Three quadruple single pole single throw switches that direct the active channel among the three available.

This multiplexing system allows for seamless switching between different sensor types and channels, enabling comprehensive temperature monitoring across multiple points.

Thermocouples

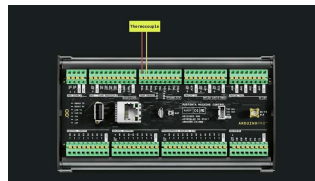
The Portenta Machine Control is compatible with **non-grounded Type K and Type J Thermocouples**.

Connect a thermocouple following the next steps:

Connect the thermocouple to one of the three channels named `TEMP_PROBES`.

Connect the thermocouple positive pin to `TPCH`.

Connect the thermocouple negative pin to `TNCH`.



Non-grounded
Thermocouple connection
to channel 0

Thermocouples can have different cable color codes depending on the region and norm. Please check the

i meaning of each cable code before connecting them to the device. Do not connect the thermocouple negative pin to GND.

The following example sketch demonstrates how to read temperatures from thermocouples connected to a Portenta Machine Control device. It uses the `Arduino_PortentaMachineControl` library to interface with the thermocouple probes and prints the temperature values to the Arduino IDE's Serial Monitor every second.

```

1  /*
2   Portenta Machine Control
3   Name: portenta_machine_control
4   Purpose: Reads temperatures from thermocouples
5   Portenta Machine Control
6   Serial Monitor every second
7
8   @author Riccardo Rizzo
9   @version 1.0 01/10/23
10 */
11
12 #include <Arduino_PortentaMachineControl.h>
13
14 void setup() {
15     // Initialize serial communication
16     Serial.begin(9600);
17
18     // Wait 2.5 seconds for initialization
19     for (const auto timeout = 2500; timeout > 0; timeout--) {
20         // Initialize temperature sensor
21         MachineControl_TCTempF.begin();
22         Serial.println("- Temp");
23     }
24 }
25
26 void loop() {
27     // Measure and display temperature
28     for (int channel = 0; channel < 4; channel++) {

```


This example sketch reads temperatures from thermocouples connected to its temperature probe inputs. It demonstrates the use of the `Arduino_PortentaMachineControl` library to interface with thermocouple sensors. The sketch initiates serial communication in the `setup()` function and then enters a loop where it reads and displays the temperature from each channel to the Serial Monitor every second.

Key functions used in the example sketch:

```
MachineControl_TCTempProbe  
.begin()
```

: Initializes the temperature probes.

```
MachineControl_TCTempProbe  
.selectChannel(channel)
```

: Selects the active channel for temperature measurement.

```
MachineControl_TCTempProbe  
.readTemperature()
```

: Reads the temperature from the selected channel.

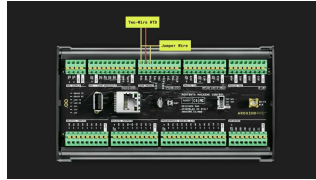
PT100

The Portenta Machine Control is compatible with two-wire RTD (PT100) and three-wire RTD (PT100) probes

Two Wire RTD Connection

The 2-wire RTD configuration is the simplest of the RTD circuit designs but is more prone to errors.

To connect a 2-wire RTD probe to one of the channels, like **channel 0**, connect one pin of the PT100 to the `TP0` input, the other pin to `TN0`, and connect a jumper between `TP0` and `PT00` pins, as you can see in the



PT100 2-wire connection to
channel 0

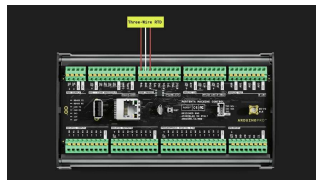


Do not connect any pin of the
PT100 to GND

Three Wire RTD Connection

The 3-wire RTD configuration is the most commonly used RTD circuit. In this configuration, two wires link the sensing element to the monitoring device on one side of the sensing element, and one links it on the other side.

To connect a 3-wire RTD probe to one of the channels, like **channel 0**, connect one of the positive wires of the PT100 to the `TP0` input, the other positive wire to `RTD0`, and the negative one to `TN0`, as you can see in the following picture.



3-wire connection to
channel 0



Do not connect any pin of the
PT100 to GND

PT100 Example

The following example sketch

probe connected to a Portenta Machine Control device. It uses the `Arduino_PortentaMachineControl` library to interface with the PT100 probes and prints the temperature values and some additional PT100 constants of the probe to the Arduino IDE's Serial Monitor every second.

```

1  /*
2  * Portenta Machine Control
3  *
4  * This example provides
5  * on the Machine Control
6  * acquire 2-wire RTDs
7  * The Machine Control
8  * which serves as a reference
9  *
10 * Circuit:
11 * - Portenta H7
12 * - Portenta Machine Control
13 * - 3-wire RTD or 2-wire RTD
14 *
15 * This example code is
16 * Copyright (c) 2024 Arduino
17 * SPDX-License-Identifier: MIT
18 */
19
20 #include <Arduino_PortentaMachineControl.h>
21
22 // The value of the Reference Temperature
23 #define RREF 400.0
24 // The 'nominal' 0-degree Celsius
25 // 100.0 for PT100
26 #define RNOMINAL 100.0
27
28 void setup() {

```

In case you want to use this example with a 2-wire RTD PT100 probe, change the function

```
MachineControl_RTDTempProbe.begin(THREE_WIRE);
```

to

```
MachineControl_RTDTempProbe.begin(TWO_WIRE);
```

This example sketch reads

inputs. It demonstrates the use of the `Arduino_PortentaMachineControl` library to interface with PT100 probes. The sketch initiates serial communication in the `setup()` function and then enters a loop where it reads and displays the temperature from each channel to the Serial Monitor every second.

Key functions used in the sketch:

```
MachineControl_RTDTempProbe.begin(THREE_WIRE)
```

: Initializes the temperature probes. In case you want to use this example with a 2-wire RTD PT100 probe, change the constant `THREE_WIRE` to `TWO_WIRE`.

```
MachineControl_RTDTempProbe.selectChannel(channel)
```

: Selects the active channel for temperature measurement.

```
MachineControl_RTDTempProbe.readRTD()
```

: Reads the raw value from the selected channel to calculate a `ratio`. The value needs to be converted to a valid temperature value in Celsius degrees using the function

```
MachineControl_RTDTempProbe.readTemperature(RNOMINAL, RREF)
```

, as can be seen in the lines that follow the calls of the function.

```
MachineControl_RTDTempProbe.readTemperature(RNOMINAL, RREF)
```

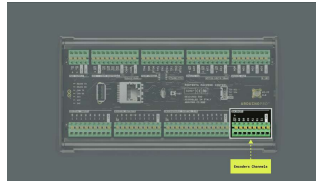
: Converts the values read from the

```
MachineControl_RTDTempProbe.readRTD()
```

function and the `RNOMINAL` constant to get the PT100 probe temperature reading in Celsius degrees.

Encoders

channels, `0` and `1`, offering precise motion control and feedback for various applications.



Portenta Machine Control
encoder channels

Some of the key capabilities of Portenta's Machine Control encoder channels are the following:

Independent operation: Each `ABZ` encoder channel functions independently, providing motion measurement and control versatility.

Pull-up resistance: The channels are equipped with 10 k Ω pull-up resistors connected to the board's 24 VDC supply, ensuring stable signal integrity and reduced noise interference.

24 VDC compatibility: The connection to the 24 VDC supply makes these encoders suitable for industrial systems and applications that operate at this voltage level.

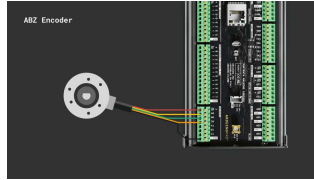
The

`Arduino Mbed OS Portenta Boards`

core and the

`Arduino_PortentaMachineControl`

have built-in libraries and functions that enable you to utilize the Portenta's Machine Control encoders. The following example shows how to read information from encoder channel 0 of a Portenta Machine Control device; for wiring an ABZ encoder to the Portenta Machine Control, follow the diagram below:



Wiring of an ABZ encoder
connected to channel 0

The example sketch is shown below.

```
1  /*
2   Portenta Machine Control
3   Name: portenta_machine_control
4   Purpose: Reads values from an encoder
5   periodically displays them
6
7   @author Riccardo Rizzi
8   @version 1.0 01/10/23
9  */
10
11 #include <Arduino_PortentaMachineControl.h>
12
13 void setup() {
14   // Initialize serial communication
15   Serial.begin(9600);
16   // Wait 2.5 seconds for the encoder to settle
17   for (const auto timeout : {2500}) {
18   }
19
20 void loop() {
21   // Read and print the encoder value
22   Serial.print("- Encoder ");
23   Serial.println(MachineControl.readEncoderValue());
24
25   // Read and print the encoder value
26   Serial.print("- Encoder ");
27   Serial.println(MachineControl.readEncoderValue());
28 }
```

This sketch for the Portenta Machine Control reads and displays data from one of its encoder channels, primarily using the

`Arduino_PortentaMachineControl`

library. Serial communication is initiated in the `setup()` function for output to the Arduino IDE's Serial

state, total pulses, and revolutions on the Serial Monitor, updating the readings every 25 milliseconds.

The sketch uses several key functions and methods:

```
MachineControl_Encoders.ge  
tCurrentState(channel)
```

: Retrieves the current binary state of the specified encoder channel.

```
MachineControl_Encoders.ge  
tPulses(channel)
```

: Obtains the total number of pulses recorded by the specified encoder channel.

```
MachineControl_Encoders.ge  
tRevolutions(channel)
```

: Calculates the total revolutions made by the specified encoder channel.

Arduino Cloud

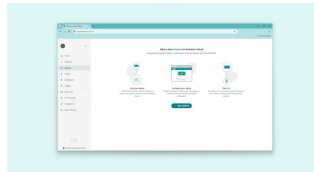
The Portenta Machine Control is fully compatible with [Arduino Cloud](#), simplifying how professional applications are developed and tracked. By using Arduino Cloud, you can, for example, monitor your Portenta's Machine Control input terminals, control your device's user LEDs and output relays remotely, and update your device's firmware OTA.

In case it is the first time you are using Arduino Cloud:

To use it, you need an account. If you do not have an account, create one for free [here](#).

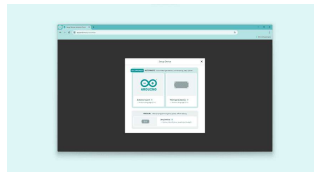
To use the Arduino Cloud Editor, the Arduino Create Agent must be running on your computer. You can install the Arduino Create Agent by downloading it [here](#).

Portenta Machine Control device with Arduino Cloud. Log in to your Cloud account; provision your Portenta Machine Control on your Cloud space. To do this, navigate to **Devices** and then click on the **ADD DEVICE** button:



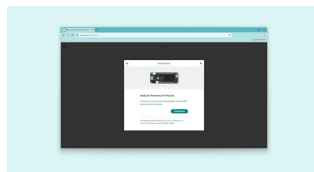
Cloud initial page

The Setup Device pop-up window will appear. Navigate into **AUTOMATIC** and select the **Arduino board** option:



Cloud Setup Device pop-up window

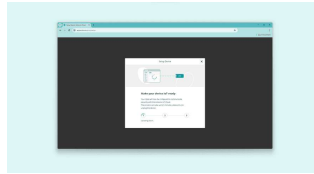
After a while, your Portenta Machine Control should be discovered by Cloud, as shown below:



Cloud Setup Device pop-up window

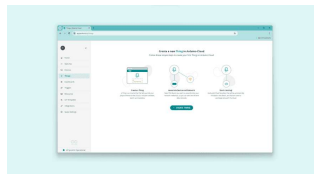
Click the **CONFIGURE** button, give your device a name, and select the type of network connection. In this example, we will use a Wi-Fi® connection; you can also use an Ethernet connection with your device. Your Portenta Machine Control will

securely with the Cloud; this process can take a while.



Cloud Setup Device pop-up window

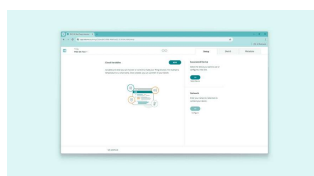
Once your Portenta Machine Control has been configured, let's create a "Thing" to test the connection between your board and Cloud. Navigate into **Things** and select the **CREATE THING** button; give your thing a name.



Cloud "Thing" setup

Navigate into **Associate Device** and click the **Select Device** button. Select your Portenta Machine Control device and associate it with your "Thing." Then, navigate into **Network** and click the **Configure** button; enter your network credentials.

The project is ready to add variables to your "Thing"; navigate into **Cloud Variables** and click the **ADD** button.



Add variable button

Add one variable with the following

Name: `led`

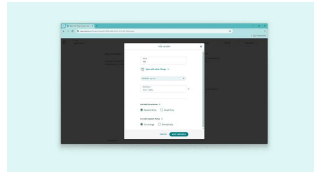
Variable type: `boolean`

Variable permission:

Read & Write

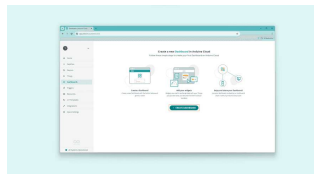
Variable update policy:

On change



Cloud "Thing" variable setup

You should see the `led` variable in the Cloud Variables section. Navigate into **Dashboards** and select the **BUILD DASHBOARD** button; this will create a new dashboard and give your dashboard a name.



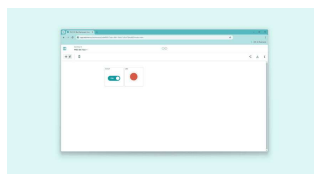
Cloud Dashboards page

Add the following widgets to your dashboard:

Switch: Name the widget Switch and link it to the `led` variable you created before.

LED: Name the widget **LED** and link it to the `led` variable you created before.

Your dashboard should look like the following:



Go back to your **Things** and open the "Thing" you created. In the "Thing" setup page, navigate into Sketch, where you should see the online editor.

In the generated sketch, include the

```
Arduino_PortentaMachineControl.h
```

library and initialize digital output at channel `0` in the `setup()` function:

```
1 #include <Arduino_PortentaMachineControl.h>
2 #include "thingProperties.h"
3
4 void setup() {
5     // Initialize serial port
6     Serial.begin(9600);
7     // This delay gives the hardware time to initialize
8     delay(1500);
9
10    // Initialize the digital output
11    MachineControl_DigitalOutputChannel(0);
12
13    // Defined in thingProperties.h
14    initProperties();
15
16    // Connect to Arduino Cloud
17    ArduinoCloud.begin(ArduinoCloud_SESSID);
18
19    /*
20     * The following function is used to set the state of the LED
21     * related to the state of the LED. The higher number the
22     * state, the higher the brightness. The default is 0 (off).
23     * The default is 0 (off). Maximum is 4
24     */
25    /*
26     * setDebugMessageLevel(2)
27     * ArduinoCloud.printDebug()
28     */
29 }
```

In the `onLedChange()` function, which was generated automatically by Cloud when the variable `led` was created, you must associate the digital output at channel `0` state with the `led` variable:

```

1  /*
2   Since Led is READ_WRITE
3   executed every time a ne
4  */
5  void onLedChange() {
6   // Change digital output
7   MachineControl_DigitalOu
8  }

```

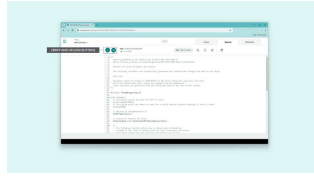
The complete example sketch can be found below:

```

1  /*
2   Sketch generated by th
3
4   Arduino Cloud Variable
5
6   The following variable
7
8   bool led;
9
10  Variables which are ma
11  which are called when
12  These functions are ge
13  */
14
15  #include <Arduino_Porter
16  #include "thingPropertie
17
18  void setup() {
19   // Initialize serial a
20   Serial.begin(9600);
21   // This delay gives th
22   delay(1500);
23
24   // Initialize the digi
25   MachineControl_Digital
26
27   // Defined in thingPro
28   initProperties();

```

To upload the code to the Portenta Machine Control from the online editor, click the green **Verify** button to compile the sketch and check for errors, then click the green **Upload** button to program the board with the



Uploading a sketch to the Portenta Machine Control in the Arduino Cloud

Navigate into **Dashboards** again; your board should connect to the Wi-Fi® network you defined before (you can follow the connection process with the online editor-integrated Serial Monitor). Your Portenta's Machine Control digital output at channel should light on or off when the position of the switch changes.

Support

If you encounter any issues or have questions while working with the Portenta Machine Control, we provide various support resources to help you find answers and solutions.

Help Center

Explore our Help Center, which offers a comprehensive collection of articles and guides for the Portenta Machine Control. The Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Portenta Machine Control
help center page](#)

Forum

users, share your experiences, and ask questions. The Forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Portenta Machine Control.

**Portenta Machine Control
category in the Arduino
Forum**

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We are happy to help you with any issues or inquiries about the Portenta Machine Control.

Contact us page

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

